OO Programozás

Sablonok, kivételkezelés

Dr. Hatwagner F. Miklós

Széchenyi István Egyetem, Győr

https://github.com/wajzy/GKxB_INTM119

2026. november 12.





Sablonok (template)

- Különböző típusokat kezelő függvények / osztályok csoportjának egyszerű létrehozása
- *Típusparaméter* (fordítási időben eldől) vs. függvény paraméter (futási időben adják át)
- Kulcsszavak:
  - Sablon → `template`
  - Típusparaméterek → `class` vagy `typename` (a sablonok *fejlécében* C++17 óta egyenértékűek; előbbit vezették be először, utóbbi kevésbé félrevezető)
- *Sablon példányosítás:* adott típusparaméterrel új változat előállítása → egyszerűbb, mint sok felültöltött függvényt kézzel létrehozni

Feladat:

Készítsünk `max` függvényt, ami két paramétere közül visszaadja a nagyobbat!

max1.cpp

```
3  // Overloaded max functions
4  int max(int a, int b) {
5    return a > b ? a : b;
6  }
7
8  double max(double a, double b) {
9    return a > b ? a : b;
10 }
```

max1.cpp

```
12 int main() {
13   const int x = 1;
14   const int y = 2;
15   std::cout << "Of " << x << " and " << y << ", "
16             << max(x, y) << " is greater.\n";
17
18   const double i = 1.5;
19   const double j = 2.5;
20   std::cout << "Of " << i << " and " << j << ", "
21             << max(i, j) << " is greater.\n";
22 }
```

Kimenet

```
Of 1 and 2, 2 is greater.
Of 1.5 and 2.5, 2.5 is greater.
```

max2.cpp

```
3  template<class T> T max(T a, T b) {
4    return a > b ? a : b;
5  }
6
7  int main() {
8    const int x = 1;
9    const int y = 2;
10   std::cout << "Of " << x << " and " << y << ", "
11             << max(x, y) << " is greater.\n";
12
13   const double i = 1.5;
14   const double j = 2.5;
15   std::cout << "Of " << i << " and " << j << ", "
16             << max(i, j) << " is greater.\n";
```

- Egy sablon több típusparamétert is fogadhat, de akkor ezeket mind használni is kell!
- Csak akkor állít elő kódot a fordító, amikor a függvény konkrét hívásával találkozik $\rightarrow$ a példányosításig a hibás sablon nem vált ki hibaüzenetet
- Pontosan olyan változatok jönnek létre, amikre szükség is van
- Típusonként pontosan egy változatot hoz létre, akkor is, ha létezik a típusok között implicit konverzió (pl. `int` $\rightarrow$ `long`)
- Mi történik, ha két *eltérő* típusú paraméterrel hívják a függvényünket?

max2.cpp

```
18    // const int k = 3;
19    // const double l = 4.5;
20    // std::cout << "Of " << k << " and " << l << ", "
21    //           << max(k, l) << " is greater.\n";
22    // error: no matching function for call to
23    // 'max(const int&, const double&)'
24    // note: candidate: template<class T> T max(T, T)
25    // note: template argument deduction/substitution failed:
26    // note: deduced conflicting types for parameter 'T' ('int' and 'double')
27  }
```

Probléma:

A két paraméter típusának mindenképpen egyeznie kell!

Félmegoldás:

Két típusparaméter használata $\rightarrow$ mindig ugyanabban a típusban (pl. `T1`) keletkezik az eredmény, akkor is, ha az nem megfelelő

max3.cpp

```
3  template<class T1, class T2> T1 max(T1 a, T2 b) {
4    return a > b ? a : b;
5  }
6
7  int main() {
8    const int i = 3;
9    const double d = 4.5;
10   std::cout << "Of " << i << " and " << d << ", "
11             << max(i, d) << " is greater.\n";
12   std::cout << "Of " << d << " and " << i << ", "
13             << max(d, i) << " is greater.\n";
14 }
```

Kimenet

```
Of 3 and 4.5, 4 is greater.
Of 4.5 and 3, 4.5 is greater.
```

Ha létezik függvény a szükséges paraméterekkel, nem jön létre példány a sablonból, és a korábbi hiba orvosolható.

max4.cpp

```
3  template<class T1, class T2> T1 max(T1 a, T2 b) {
4    return a > b ? a : b;
5  }
6
7  double max(int a, double b) {
8    return a > b ? a : b;
9  }
```

Kimenet

```
Of 3 and 4.5, 4.5 is greater.
Of 4.5 and 3, 4.5 is greater.
```

Problémák:

- A felültöltött függvények létrehozását szerettük volna elkerülni a sablonozással
- Könnyen kifelejthető egy szükséges függvény létrehozása → hibaforrás

C++14: fordítási időben megállapítható tetszőleges típusok közös típusa, amibe bármelyik konvertálható, pl. `std::common_type_t<int, double>` → `double`

max5.cpp

```
2 #include <type_traits>
3
4 template<class T1, class T2>
5 std::common_type_t<T1, T2> max(T1 a, T2 b) {
6   return a > b ? a : b;
7 }
```

C++11-től kezdve egyre több helyen (pl. függvény visszatérési értékének típusa, változók deklarálása) alkalmazható az `auto` kulcsszó a konkrét típus helyőrzőjeként → fordító megpróbálja kikövetkeztetni a megfelelő típust. A példában konkrétan a feltételes operátor típuspromóciós szabályai alapján a kifejezés típusa a tágabb típus lesz → egyszerűbb használat (nem kell `include` se), de feltételezi a függvény testének megvizsgálhatóságát.

max6.cpp

```
1 #include <iostream>
2
3 template<class T1, class T2>
4 auto max(T1 a, T2 b) {
5   return a > b ? a : b;
6 }
```

Fontosabb tudnivalók:

- A függvénysablonokhoz hasonlóan *osztálysablonokat* is létrehozhatunk.
- A típusparaméterek mellett *konstansparamétereket* (non-type template parameter, NTTP) is megadhatunk $\rightarrow$ bárhova helyettesíthetők, ahol konstans kifejezés állhat.
- Ha egy tagfüggvényt az osztályon kívül definiálunk, meg kell ismételni a `template` kulcsszót a paraméterekkel együtt. A definíciónak kivételesen a fejfájlban a helye, hogy a fordító mindig elérje.
- Osztálysablon példányosítása: a típus után < és > között meg kell adni a konkrét típusokat, konstansokat. Csak ezek együtt azonosítják egyértelműen a sablon alapján előállított osztályt!

Stack1.hpp

```
6  template<class T, int I> class Stack {
7      T array[I];
8      int used;
9    public:
10     Stack() : used(0) {}
11     void push(T data);
```

Stack1.hpp

```
12      T pop() {
13        if(used > 0) {
14          return array[--used];
15        } else {
16          std::cerr << "Oops, the stack is empty.\n";
17          return array[0]; // something must be returned
18        }
19      }
20      bool isEmpty() const {
21        return used == 0;
22      }
23  };
```

Stack1.hpp

```
25 // It MUST be in the header!
26 template<class T, int I> void Stack<T, I>::push(T data) {
27   if(used < I) {
28     array[used++] = data;
29   } else {
30     std::cerr << "Stack full :(\n";
31   }
32 }
```

stackMain1.cpp

```
1  #include <iostream>
2  #include "Stack1.hpp"
3
4  int main() {
5    Stack<int, 3> si;
6    si.push(1); si.push(2); si.push(3);
7    si.push(4); // stack full
8    while(not si.isEmpty()) {
9      std::cout << si.pop() << std::endl;
10   }
```

stackMain1.cpp

```
12    Stack<const char*, 10> scp;
13    scp.push("World!\n"); scp.push("Hello ");
14    std::cout << scp.pop();
15    std::cout << scp.pop();
16    std::cout << scp.pop(); // stack empty
17  }
```

Kivételkezelés (*exception handling*)

- *Kivételes*, azaz ritka események (hibák) kezelésére
- A standard könyvtár vagy saját kód is kiválthatja, pl. `new` → `std::bad_alloc`
- De pl. egy tömb túlindexelése nem definiált viselkedés (undefined behavior, UB), a nullával osztás pedig processzormegszakítás (`SIGFPE`) → kivételkezelés nem megoldás
- Védett blokk: felkészülés kivételes helyzet bekövetkezésére → `try`
- Kivétel kiváltása → `throw`
- Kivétel elfogása → `catch`

`try` blokkok jellemzői:

- Egymásba ágyazhatók.
- Benne közvetlenül vagy közvetetten (függvényen belül) is használható `throw` utasítás.
- Legalább egy, de akár több `catch` ág is tartozhat egy `try`-hoz.

`throw` jellemzői:

- Tetszőleges típusú kivétel kiváltható.
- A `try` blokk végrehajtása azonnal félbeszakad a hatására.
- Használható akár konstruktorban, vagy `catch` blokkban is.
- A `throw;` utasítás a korábban kiváltott és már elfogott típusú és tartalmú kivétellel azonosat vált ki újra.

`catch` blokkok jellemzői:

- Az első illeszkedő blokk kezeli a kivételt.
- Nyilvánosan származtatott kivétel osztályok esetén a szülő típusára illeszthető a leszármazott típusú kivétel → speciálisak előre, általánosabbak hátra!
- Nem illeszkednek viszont az implicit típuskonverzióval egyébként egymásba alakítható típusok `catch` blokkjai (pl. `int` típusú kivételt nem fogja el a `float` kezelője).
- A speciális, `...` típusú `catch` minden, korábban nem kezelt kivételt elfog → ez legyen az utolsó!
- A kivétel kezelését követően a program fut tovább az utolsó `catch` blokk után.
- Ha nincs illeszthető `catch` ág, a vezérlés a befoglaló `try-catch` szerkezetre kerül, és ott keres illeszkedő kivételkezelőt. Ha ilyen nincs, a vezérlés a `terminate()` függvényre kerül, és a program leáll.

Veremlebontás (stack unwinding)

Elhagyott blokkok lokális objektumainak destruktorai automatikusan lefutnak, RAII elvű objektumok felszabadítják erőforrásaikat, emiatt

- nincs szükség `finally` blokkokra, mint pl. Java, C# nyelvekben
- *destruktorból tilos kivételt dobni* (C++11-től implicit `noexcept`)

Kivételkezelési stratégiák:

1. Figyelmeztetés, felhasználói beavatkozás kérése, megpróbálkozás a hiba javításával.
2. Program leállítása (pl. `exit()`, `abort()`) súlyos hibák esetén.
3. Kivétel kezeletlenül hagyása.

trycatch1.cpp

```
1  #include <iostream>
2
3  class BaseException {};
4  class DerivedException : public BaseException {};
5  // private inheritance --> exception object's reference
6  //                         cannot be bound to base reference
7
8  int main() {
9    for(int exceptType = 0; exceptType < 6; exceptType++) {
10     try {
11       switch(exceptType) {
```

trycatch1.cpp

```
12          case 0:
13            std::cout << "Throwing int... ";
14            throw 42;
15            break;
16          case 1:
17            std::cout << "Throwing double... ";
18            throw 3.14;
19            break;
20          case 2:
21            std::cout << "Throwing const char*... ";
22            throw "Can you access it?";
23            break;
```

trycatch1.cpp

```
24          case 3:
25            std::cout << "Throwing␣BaseException...␣";
26            throw BaseException();
27            break;
28          case 4:
29            std::cout << "Throwing␣DerivedException...␣";
30            throw DerivedException();
31            break;
32          case 5:
33            std::cout << "Throwing␣char...␣";
34            throw 'x';
35            break;
36        }
```

trycatch1.cpp

```
37      } catch(char c) {
38        std::cout << "caught " << c << ", re-throw exception"
39                  << std::endl;
40        throw;
41      // no implicite type conversions, no conversion ctor calls
42      } catch(double d) {
43        std::cout << "caught " << d << std::endl;
44      } catch(const char*) { // parameter without name
45        std::cout << "caught an unknown message" << std::endl;
```

trycatch1.cpp

```
46      // warning: exception of type 'DerivedException' will be
47      //          caught by earlier handler for 'BaseException'
48      } catch(const BaseException& se) {
49        std::cout << "caught BaseException object" << std::endl;
50      } catch(const DerivedException& de) {
51        std::cout << "caught DerivedException object" << std::endl;
52      } catch(...) {
53        std::cout << "caught an UNKNOWN exception" << std::endl;
54      }
55    }
56  }
```

Kimenet

```
Throwing int... caught an UNKNOWN exception
Throwing double... caught 3.14
Throwing const char*... caught an unknown message
Throwing BaseException... caught BaseException object
Throwing DerivedException... caught BaseException object
Throwing char... caught x, re-throw exception
terminate called after throwing an instance of 'char'
Aborted (core dumped)
```

trycatch2.cpp

```
4  void fn() {
5    try {
6      std::cout << "Do you want to throw an exception of type int? ";
7      std::string answer;
8      std::cin >> answer;
9      if(answer == "y") throw 1;
10     throw 1.2;
11   } catch(double) {
12     std::cout << "Caught a double.\n";
13   }
14 }
```

trycatch2.cpp

```
16 int main() {
17   try {
18     fn();
19     try {
20       std::cout << "Do you want to throw an exception of type char? ";
21       std::string answer;
22       std::cin >> answer;
23       if(answer == "y") throw 'a';
24       throw "message";
25     } catch(char) {
26       std::cout << "Caught a char.\n";
27     }
28   } catch(int) {
29     std::cout << "Caught an int.\n";
30   } catch(const char*) {
31     std::cout << "Caught a pointer to char.\n";
32   }
33 }
```

Kimenet1

```
Do you want to throw an exception of type int? y
Caught an int.
```

Kimenet2

```
Do you want to throw an exception of type int? n
Caught a double.
Do you want to throw an exception of type char? y
Caught a char.
```

Kimenet3

```
Do you want to throw an exception of type int? n
Caught a double.
Do you want to throw an exception of type char? n
Caught a pointer to char.
```

Alakítsuk át a verem megvalósítását úgy, hogy kivételkezelést használjon! Használjuk fel a függvénykönyvtár `exception` osztályát!

Stack2.hpp

```
4  #include <exception>
5  #include <iostream>
6  #include <string>
7
8  class StackException : public std::exception {
9    public:
10     enum ErrorCode { UNDEFINED, EMPTY, FULL };
11   private:
12     ErrorCode code;
13     std::string message;
```

Stack2.hpp

```
14    public:
15      StackException() : code(ErrorCode::UNDEFINED), message("") {}
16      StackException(ErrorCode code, const std::string& message) :
17        code(code), message(message) {}
18      ErrorCode getCode() const {
19        return code;
20      }
21      // we have to promise not to throw exceptions in what()
22      virtual const char* what() const noexcept {
23        return message.c_str();
24      }
25  };
```

Stack2.hpp

```
27  template<class T, int I> class Stack {

30    public:

33      T pop() {
34        if(used > 0) {
35          return array[--used];
36        } else {
37          throw StackException(
38            StackException::ErrorCode::EMPTY,
39            "Oops, the stack is empty."
40          );
41        }
42      }

46  };
```

Stack2.hpp

```
48 template<class T, int I> void Stack<T, I>::push(T data) {
49   if(used < I) {
50     array[used++] = data;
51   } else {
52     throw StackException(
53       StackException::ErrorCode::FULL,
54       "Sorry, stack full."
55     );
56   }
57 }
```

stackMain2.cpp

```
4  int main() {
5    Stack<int, 3> si;
6    try {
7      si.push(1); si.push(2); si.push(3);
8      si.push(4);
9    } catch(const StackException& se) {
10     std::cerr << "Error code: " << se.getCode()
11               << ", " << se.what()
12               << std::endl;
13   }
14   while(not si.isEmpty()) {
15     std::cout << si.pop() << std::endl;
16   }
```

stackMain2.cpp

```
18   Stack<const char*, 10> scp;
19   scp.push("World!\n"); scp.push("Hello ");
20   try {
21     std::cout << scp.pop();
22     std::cout << scp.pop();
23     std::cout << scp.pop();
24   } catch(const StackException& se) {
25     std::cerr << "Error code: " << se.getCode()
26               << ", " << se.what()
27               << std::endl;
28   }
29 }
```

Kimenet

```
Error code: 2, Sorry, stack full.
3
2
1
Hello World!
Error code: 1, Oops, the stack is empty.
```